

SILK: Constraint-guided Hybrid Fuzzing

Junhao Li ^{*}, Yujian Zhang ^{*†‡},

^{*} School of Cyber Science and Engineering, Southeast University, Nanjing, China

[†] Jiangsu Province Engineering Research Center of Security for Ubiquitous Network, Nanjing, China

[‡] Corresponding author: yjzhang@seu.edu.cn

Abstract—Hybrid fuzzing combines fuzzing and concolic execution which leverages the high-throughput feature of fuzzing to explore easy-to-reach code, and the powerful constraint solving capability of concolic execution to explore code wrapped in complex constraints. Based on our observations, existing hybrid fuzzers are still not efficient for the following two reasons. First, fuzzing often gets stuck in deep paths leading to the delayed discovery of vulnerabilities. Second, coarse-grained interaction strategies cannot effectively launch concolic execution. To solve the above problems, we propose a constraint-guided hybrid fuzzing approach (CGHF) that leverages the constraints’ static analysis information and dynamic execution information. CGHF contains two main techniques: an evolutionary algorithm based on path exploration difficulty and an interaction strategy guided by the execution state of constraints. Specifically, in the fuzzing phase, we evaluate the path exploration difficulty and guide the fuzzer to explore in the order of difficulty from low to high. In addition, we design a coordinator to monitor the constraints’ dynamic execution information and select the most deserving constraints to be solved for the concolic execution. We implement a prototype called SILK and compare its effectiveness on eight open source programs with other state-of-the-art fuzzers. The results show that SILK improved path coverage by 10%-45% and branch coverage by 5%-10% compared with other fuzzers.

Index Terms—fuzzing, concolic execution, hybrid fuzzing, constraint

I. INTRODUCTION

Evolutionary mutation-based fuzzing (e.g., AFL) has become one of the most widely used techniques for vulnerability discovery. Such fuzzers are difficult to find vulnerabilities hidden in deep paths of the programs because of the randomness of mutation. To improve the performance of fuzzing, some researchers combine both fuzzing and concolic execution hoping that the concolic execution will help the fuzzing solve complex constraints. This technique, called hybrid fuzzing, has shown its effectiveness in real-world programs and various synthetic benchmarks. However, based on our observations, existing hybrid fuzzing still incurs inefficiency due to the following two issues.

First, the coverage-based evolutionary algorithm delays the discovery of vulnerabilities by hybrid fuzzing. Fuzzing uses the coverage-based evolutionary algorithm to allocate more energy for seeds that can discover new paths, increasing the probability of generating new seeds that can pass complex constraints. The algorithm leads to a preferential exploration of deep execution paths and delay of some easy-to-reach code. However, in hybrid fuzzing, fuzzing can take advantage of the ability of concolic execution to solve complex constraints without the help of evolutionary algorithm. The main task of

fuzzing is to take advantage of its high throughput to quickly discover constraints in the target program, rather than getting stuck in solving nested constraints.

Second, the coarse-grained interaction strategy cannot efficiently launch concolic execution. There are two main types of interaction strategies. One is the “demand launch” strategy, the other is the “path priority” strategy. Driller [1] adopts the “demand launch” strategy that takes the execution state of fuzzing as the signal to launch concolic execution. Since the signal is coarse-grained, even if the fuzzer has gotten stuck on some constraints, the concolic executor will be idle as long as the fuzzer can still discover new paths. To better combine fuzzing and concolic execution, some work [2] [3] [4] has proposed the “path priority” strategy. However, these researches face problems such as high cost of time and inaccurate estimation of the properties of paths.

To solve the above problems, we propose a novel effective hybrid fuzzing technique CGHF, which guides hybrid fuzzing with constraints’ static structural information and dynamic execution information. Specifically, in contrast to coverage-based fuzzing which prefers to explore deep paths, in our work, to take full advantage of the high-throughput nature of fuzzing, we let the fuzzer explore easy-to-reach paths first and then hard ones. Constraints’ static structural information is used to distinguish between easy and hard paths. After the fuzzing starts, we monitor the execution state of constraints and use the stuck state of constraints as the indicator to decide whether the concolic execution is needed. Specifically, the number of executions of constraints’ different branches is used as the basis for determining the stuck state. Concolic execution preferentially helps fuzzing solve constraints that significantly differ in the number of executions of different branches. By analyzing the execution state of constraints, concolic execution can find the constraint that is most worth solving (i.e., the constraint that, when solved, can best help fuzzing improve code coverage).

We have implemented the CGHF approach in a tool called SILK based on the well-known fuzzer, American Fuzzy Lop (AFL), and the light-weight concolic executor QSYM [5]. SILK is evaluated on eight real-world programs. Our evaluation results show that SILK improves path coverage by 10%-45% and branch coverage by 5%-10% compared with other state-of-the-art fuzzers.

In summary, the contributions of our work can be summarized as follows:

- We propose an evolutionary algorithm based on path

exploration difficulty that can take full advantage of the high-throughput nature of fuzzing.

- We propose a fine-grained interaction strategy guided by the execution state of constraints which can efficiently launch concolic execution.
- We open source our implementation SILK on <https://github.com/SEU-SSL/SILK>, and evaluate it on eight real-world programs. SILK achieves significantly higher path and branch coverage than other state-of-the-art fuzzers.

II. BACKGROUND

A. Coverage-based Fuzzing

Coverage-based fuzzing is widely used and has been proved effective. The core idea is to use whether the seed can improve code coverage as an indicator to guide the direction of mutation [6]. The workflow of coverage-based fuzzing can be summarized as follows.

- 1) Construct an initial seed queue that is used for mutation.
- 2) Select a queued seed and mutate it with several kinds of mutation operators.
- 3) Employ the generated test cases to test the target program and trace their code coverage.
- 4) If a test case can improve code coverage or trigger abnormal behaviors, add it into the seed queue, and prioritize it for the next round of mutation.
- 5) Go back to step 2 and repeat.

Due to the randomness of mutation operations, mutation-based fuzzing cannot reach deep paths with complex constraints. The coverage-based evolutionary algorithm is used to guide the direction of mutation by giving higher priorities to seeds that can discover new paths. However, the difference in constraint solving capabilities between fuzzing and hybrid fuzzing makes the algorithm a poor match for hybrid fuzzing. More details will be given in Section III-A.

B. Hybrid Fuzzing

To improve the constraint solving capability of coverage-based fuzzing, hybrid fuzzing combines fuzzing and concolic execution. Concolic execution is a program testing technique that is good at finding inputs that can drive the program into complex branch conditions, but it has a high time overhead to solve constraints. The workflow of hybrid fuzzing is shown in Figure 1 and can be summarized as follows.

- 1) Fuzzer tests the target program with new seeds generated by mutation and saves interesting seeds.
- 2) Coordinator determines when to start the concolic executor and which seeds to pass from the fuzzer to the concolic executor based on the interaction strategy.
- 3) Concolic executor performs constraint solving on the incoming seeds, and passes the new seeds to the fuzzer.
- 4) Fuzzer and concolic executor continuously exchange seeds and export the seeds that can trigger crashes.

Hybrid fuzzing consists of three parts, namely fuzzer, coordinator and concolic executor. In addition to the algorithmic improvements in fuzzer, another concern in our work is the

interaction strategy of the coordinator. Existing interaction strategies face problems such as coarse granularity and low efficiency. More details will be given in Section III-B. It is worth mentioning that improvement to concolic execution is a very important research direction in hybrid fuzzing, and our work is compatible with such work [5] [7].

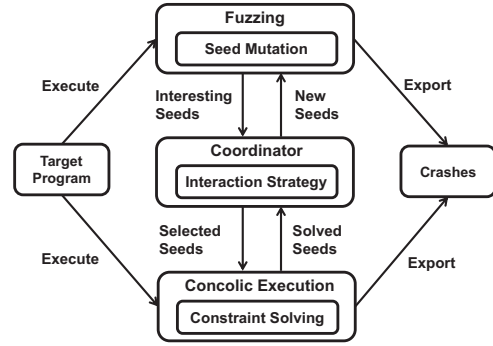


Fig. 1. The workflow of hybrid fuzzing.

III. MOTIVATION

A. Inefficiency in coverage-based evolutionary algorithm

Consider the program shown in Listing 1, which has four complex constraints (i.e., *constraint A* - *constraint D*). Fuzzing is difficult to pass these complex constraints due to its weak constraint solving ability and executes the path shown in Figure 2(a). Then the concolic executor will help the fuzzer solve *constraint A* and pass newly generated seeds to the fuzzer. Since newly generated seeds can help the fuzzer explore the new path shown in Figure 2(b), the coverage-based evolutionary algorithm will give the seeds higher priorities. Therefore, the fuzzer tends to explore the True branch of *Constraint A* (i.e., the path shown in Figure 2(c)). Since there are nested constraints on the path shown, the fuzzer will get stuck again and ask for help with the concolic executor. The interaction between fuzzing and concolic execution costs a lot of time.

In this example, the most concerning part of the program should be *constraint D* since there is a vulnerability, and it only takes one interaction with the concolic execution to trigger the vulnerability. However, the coverage-based evolutionary algorithm prefers to explore deep paths leading to the neglect of the *constraint D*. In other words, not all vulnerabilities are in deep paths, and prioritizing the execution of deep paths can cause a delay in discovering vulnerabilities [8]. Since concolic execution in hybrid fuzzing can effectively solve constraints, there is no need to use the coverage-based evolutionary algorithm to guide the direction of mutation to pass complex constraints and reach deep paths. To address the above problem, we propose an evolutionary algorithm based on the difficulty of path exploration which prioritizes the exploration of easy-to-reach paths and then deep paths to take full advantage of the high throughput of fuzzing. We will describe the algorithm in detail in Section IV-A.

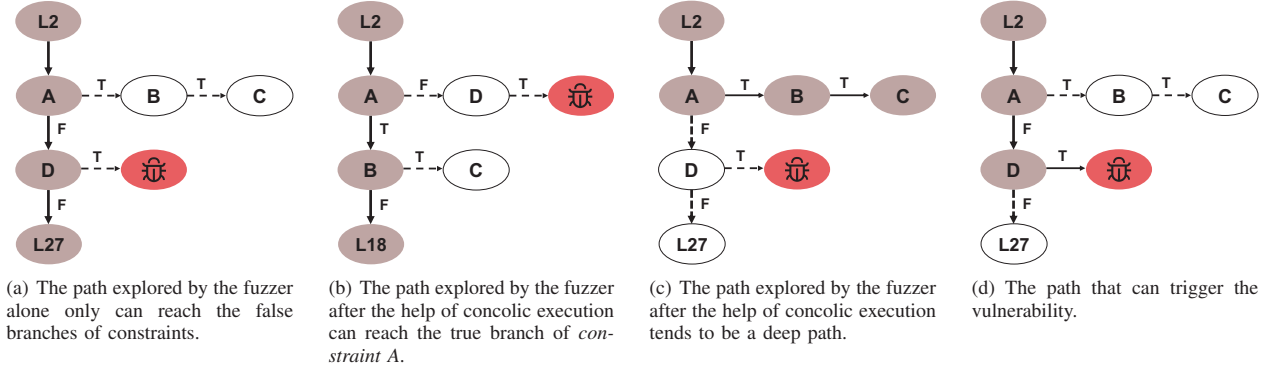


Fig. 2. Paths that can be explored in the program shown in Listing 1. The boxes connected by solid arrows are covered code, other boxes are uncovered code. Red boxes represent vulnerabilities. L2, L18, L27 represent line 2, line 18 and line 27 of the program respectively. A, B, C, D represent Constraint A, Constraint B, Constraint C and Constraint D respectively.

```

1 #define SIZE 1000
2 int main(int argc, char **argv){
3     unsigned int data[SIZE];
4     unsigned char input[SIZE];
5     //Constraint A
6     if(input[0] == 0xAB){
7         NormalCode();
8         //Constraint B
9         if(input[1] == 0xEF &&
10            data[0]*data[0] < 100){
11             //Constraint C
12             if(data[0] > 8){
13                 NormalCode();
14                 return 0;
15             }
16             NormalCode();
17         }
18         ComplexCode();
19         return 0;
20     }
21     //Constraint D
22     if(input[0] == 0xFD){
23         //Vulnerability
24         ErrorCode();
25         return 0;
26     }
27     return -1;
28 }

```

Listing 1. A motivation example.

B. Inefficient interaction strategy between fuzzing and concolic execution

In general, existing interaction strategies can be divided into the following two categories.

1) **Demand Launch:** By examining how Driller works on binaries from DARPA Cyber Grand Challenge (CGC) [9], we summarize two limitations of the “demand launch” strategy.

- Coarse-grained launch signal leads to delayed invocation of concolic execution. Driller starts fuzzing first and determines whether to invoke concolic execution by examining execution states of fuzzing. Specifically, when the *pending_favs* attribute which can be found in the *fuzzer_stats* file is greater than 0, Driller assumes that fuzzing can still find new paths, thus does not need the

help of the concolic execution. The concolic executor will be idle until *pending_favs* becomes 0. However, based on the observations in [4], even if *pending_favs* is greater than 0, fuzzing has stuck on some hard paths and needs the help of concolic execution. Thus, the execution state of fuzzing is not an accurate indicator of whether concolic execution is required.

- Concolic executor sequentially solves the seeds passed by the fuzzer leading to a reduced probability of helping the fuzzer to solve key constraints. Due to the significant gap in throughput between fuzzing and concolic execution, the concolic executor cannot execute all the seeds retained by fuzzing. Note that over time, the number of seeds retained by the fuzzer will get larger and larger than those executed by the concolic executor. This leads to new paths found by concolic execution that may have already been discovered by fuzzing. What’s worse is that the possibility keeps growing.

2) **Path Priority:** MDPC [3], DigFuzz [4] and QuickFuzz [2] adopt the “Path Priority” strategy to improve the performance of hybrid fuzzing, but they still have certain limitations. Specifically, MDPC attempts to accurately estimate the cost of exploring each path through fuzzing and concolic execution and faces the problems of inaccurate metrics and high computational overhead. DigFuzz utilizes the *MCP*³ model to let concolic execution solve the hardest paths for fuzzing, but the model ignores the cost of solving constraints and requires a large number of random sampling. QuickFuzz evaluates each path’s solving cost and solving demand by path lengths and sample hits of basic blocks. However, QuickFuzz has not proved its effectiveness in real-world programs, and cannot prove that there is an exact correspondence between path length and solving cost in real-world programs.

The “path priority” strategy focuses on prioritizing paths, hoping to find the most cost-effective path for concolic execution. However, there are two limitations of using path priority to select the seeds passed to the concolic execution. First, due to the low efficiency of constraint solving, the concolic execution takes a lot of time to explore a complete

path. To ensure efficient generation of new seeds, concolic execution (e.g., QSYM [5]) returns new seeds to the fuzzing as soon as a new constraint is found to have been solved rather than after exploring a complete path. Therefore, even if a path is prioritized, there is limited guidance for concolic execution. Second, in order to make the optimal choice, MDPC needs numerous calculations. To represent the testing process, DigFuzz and QuickFuzz need to construct execution trees that require additional large storage space. These additional overheads reduce the overall throughput of hybrid fuzzing.

IV. CONSTRAINT-GUIDED HYBRID FUZZING

In this section, we describe the design of SILK and discuss its key techniques. In order to solve the problems mentioned in Section III and improve the performance of hybrid fuzzing, we propose two key techniques: **path exploration difficulty-based evolutionary algorithm** and **constraints execution state-guided interaction strategy**. We will introduce these two techniques in Section IV-A and Section IV-B respectively. At a high level, SILK is a constraint-guided hybrid fuzzer that consists of a fuzzer, coordinator, and concolic executor. The detailed system design of SILK will be introduced in Section IV-C.

A. Path exploration difficulty-based evolutionary algorithm

Fuzzers in classic hybrid fuzzing directly apply the coverage-based evolutionary algorithm, resulting in the delayed discovery of vulnerabilities. To improve the efficiency of discovering vulnerabilities, SILK uses the path exploration difficulty-based evolutionary algorithm which prioritizes the exploration of easy-to-reach paths (i.e., breadth-first exploration) and then deep paths. Breadth-first exploration allows fuzzers to explore more easy-to-reach paths and expose more constraints in a short time. Concolic execution can then help fuzzing solve constraints on easy-to-reach paths in a more timely manner. However, implementing the breadth-first approach has two main challenges as follows.

C1: Estimating path exploration difficulty. Estimating the difficulty of exploring each path is a critical step that plays an important role in the overall effect of the algorithm. Note that the path exploration difficulty in SILK is for fuzzing which is different from other work. For example, MDPC [3] needs to accurately evaluate the difficulty of exploring a path by fuzzing and concolic execution separately before determining which technique is better for exploring the path. Since SILK only considers the difficulty of exploring each path by fuzzing, it does not need to precisely evaluate the exploration difficulty, but only needs to be able to distinguish different paths. Due to the random mutation nature of fuzzing, it is difficult for fuzzing to pass complex constraints. Therefore, the number of constraints on paths and the complexity of constraints determine the difficulty of exploring paths. There are two main methods to analyze the complexity of constraints, including direct analysis [10] [7] [11] and estimation using statistical models [12] [2]. Both methods face the problem of balancing efficiency and accuracy. To avoid this problem, SILK estimates

the exploration difficulty only by the number of constraints on the paths. Specifically, the higher the number of constraints on a path is, the harder the path is for fuzzing to explore. This evaluation method not only serves for distinguishing paths, but also mitigates computational overhead. We will describe the method in detail later.

C2: Guiding fuzzing exploration direction. In fuzzing, although seed mutation is random, the number of seed mutations (i.e., power schedule [13]) and the selection of new seeds can be controlled. Seeds with more energy and newly retained seeds are more likely to dominate the direction of fuzzing. Specifically, initial seeds are mutated to generate many new seeds. During the mutation phase, if seeds with low path exploration difficulty are given more chances to mutate, the fuzzer will generate more new seeds that can explore easy-to-reach paths. On the other side, if only new seeds with low path exploration difficulty are selected, the fuzzer will tend to explore further on easy paths. Both steps in fuzzing described above affect the exploration direction. However, guiding exploration direction by seed selection requires a significant amount of time to evaluate the path exploration difficulty. To maintain the high-throughput characteristics of fuzzing, SILK guides the exploration direction of fuzzing through power scheduling. Power scheduling only needs to evaluate the path difficulty of retained seeds [6] (i.e., seeds considered interesting by the fuzzer) and has almost no impact on the throughput of fuzzing.

A constraint is represented as a source basic block with two outgoing edges (i.e., branch edges) in the control flow graph. Each branch corresponds to either a True edge or a False edge of the constraint, which is denoted as T and F, respectively [14]. Two branch edges with the same source basic block are called sibling edges (denoted as SE). Before starting fuzzing, SILK uses a lightweight static analysis to (i) obtain the set of branch edges (denoted as BE) by scanning the binary code of the target program [15], and (ii) compute the score for each branch edge (denoted as ES). The edge score represents the sum of the exploration difficulty of the paths that the edge can reach. Since we use the number of constraints on the path to evaluate the path difficulty, the edge score refers to the sum of the number of constraints on the paths that can be reached by the edge. Take the code in Listing 1 as an example, the True branch edge of *constraint A* can reach *constraint B* and *constraint C*. Therefore, the edge score of the True branch of *constraint A* is 2. The False branch of *constraint A* can only reach *constraint D*. Therefore, the edge score of the False branch of *constraint A* is 1.

Algorithm 1 describes how SILK calculates seed power. The algorithm accepts four inputs: (i) *EE*: edges explored by the seed. (ii) *BE*: branch edges in the target program. (iii) *SE*: correspondence of the sibling edges. (iv) *ES*: score of each branch edge. When a seed is executed by the fuzzer, SILK first traverses the edges explored by the seed to find branch edges and then finds the corresponding sibling edges through *SE*. If the sibling edge corresponding to a branch edge is not explored, SILK finds the score of the sibling edge in the static

analysis result ES and records it in the edge score set. The scores of the unexplored sibling edges determine the score of the seed. Specifically, If a sibling edge is not explored by the seed, the paths reachable by this edge are potentially reachable by the new seeds generated by mutation. Therefore, the sum of the exploration difficulty of the reachable paths of the unexplored sibling edges is part of the exploration difficulty of the unexplored paths of the seed. To find the minimum difficulty of the unexplored paths of the seed, SILK takes the minimum value of $edgeScore$ as the $seedScore$. When calculating seed power, SILK uses the simulated annealing algorithm proposed by AFLGo [13], which gradually assigns more power to seeds with lower seed scores, while reducing the power for seeds with higher exploration difficulty.

Algorithm 1: Power scheduling algorithm in SILK

Input: EE , a set of edges explored by the seed
 BE , a set of branch edges
 SE , correspondence of the sibling edges
 ES , the scores of branch edges

Output: $seedPower$

```

foreach  $edge_i \in EE$  do
  if  $edge_i \in BE$  then
     $sibling\ edge_i \leftarrow FindEdge(SE)$ ;
    if  $sibling\ edge_i \notin EE$  then
       $edgeScore_i \leftarrow FindScore(sibling\ edge_i)$ ;
    end if
  end if
end foreach
 $seedScore = \min \{edgeScore_1, \dots, edgeScore_n\}$ ;
 $seedPower = CalculatePower(seedScore)$ ;
return  $seedPower$ 

```

We use the example mentioned in Section III-A to demonstrate the effect of Algorithm 1. The seed that follows the path shown in Figure 2(a) is denoted as $seed_1$. The True branch of *constraint A* (denoted as CAT) and the True branch of *constraint D* (denoted as CDT) are unexplored. The edge score of CAT is 2 which is calculated in the above example. Since CDT cannot reach any constraint, the score of CDT is 0. The score of $seed_1$ is the smaller value of the edge score of CDT and the edge score of CAT, which is 0. The seed that follows the path shown in Figure 2(b) is denoted as $seed_2$. The False branch of *constraint A* (denoted as CAF) and the True branch of *constraint B* (denoted as CBT) are unexplored. The edge score of CAF is 1 which is calculated in the above example. Since CBT can reach constraint C, the edge score of CBT is 1. Therefore, the score of $seed_2$ is 1. The score of $seed_1$ is less than $seed_2$, which means that the minimum exploration difficulty of the unexplored paths of $seed_1$ is less than that of $seed_2$. During the power scheduling phase, SILK will give $seed_1$ more mutation times, thus there will be a greater probability to generate seeds that can execute the path shown in Figure 2(d).

B. Constraints execution state-guided interaction strategy

The fundamental difference between fuzzing and concolic execution is the difference in the ability to solve constraints. In hybrid fuzzing, the role of the concolic execution is to help fuzzing solve complex constraints and increase the probability of reaching deep paths. Therefore, complex constraints are what concolic execution should be most concerned about. Note that fuzzing has a probability of passing complex constraints by random mutation, although the probability is low. Obviously, constraints that have already been solved by fuzzing do not require concolic execution to help solve them. Therefore, whether each constraint in fuzzing is solved or not is also a concern for concolic execution. In summary, constraint complexity and constraint solving results are key factors affecting the effectiveness of the interaction strategy. As mentioned in Section IV-A, existing methods for analyzing the complexity of constraints face inaccuracy and inefficiency problems. In addition, different fuzzers have different capabilities of solving constraints due to different internal algorithms. Defining a uniform metric for all fuzzers to evaluate the complexity of the constraints is a difficult and onerous task. For the purposes of light weight and scalability, our work evaluates the complexity of constraints with dynamic execution information rather than static analysis. Therefore, we propose an interaction strategy guided by the constraint execution state that can identify complex constraints and monitor the results of constraint solving.

We classify the execution states of constraints into three categories: *unreached*, *unsolved* and *solved*. Specifically, a constraint in a control flow graph consists of three parts: a basic block, True branch, and False branch. As listed in Table I, if both branch edges of a constraint are not executed, we consider the constraint is in the *unreached* state. If either of the two branch edges of a constraint is executed, we consider the constraint is in the *unsolved* state. If both branch edges of a constraint are executed, we consider the constraint is in the *solved* state. For example, given seeds that only execute the path in Figure 2(a), *Constraint A* is in the *unsolved* state since *Constraint A* only executes the False branch, *Constraint B* is unreachable because it is on the True branch of *Constraint A*. Assuming that, with the increasing fuzzing time or the help of concolic execution, some newly generated seeds can execute the path in Figure 2(b), *Constraint A* transforms from the *unsolved* state to the *solved* state since both branches of *Constraint A* are executed. *Constraint B* transforms from the *unreached* state to the *unsolved* state since the False branch of *Constraint B* is executed, but the True branch of *Constraint B* is not executed. In our work, we focus on constraints that are in the *solved* and *unsolved* state and omit constraints in the *unreached* state. Because if a constraint in the *unsolved* state is solved, the fuzzer can explore the previously unexecuted branch of the constraint and discover new paths. However, if a constraint is unreachable, even if solved, the fuzzer still cannot execute any of its branches due to the limitations of the previous paths. Although solved constraints cannot help

fuzzing to discover new paths, they can be used to indicate rare paths.

TABLE I
CONSTRAINTS EXECUTION STATE

State	True	False	Priority Score
Unreached	✗	✗	0
Unsolved	✓	✗	$\sum_{i=1}^n T(seed_i)$
Unsolved	✗	✓	$\sum_{i=1}^n F(seed_i)$
Solved	✓	✓	$\frac{ \sum_{i=1}^n T(seed_i) - \sum_{i=1}^n F(seed_i) }{\sum_{i=1}^n T(seed_i) + \sum_{i=1}^n F(seed_i)}$

To identify the execution state of constraints, we record the number of executions of each branch of constraints. Specifically, we maintain a statistic table about the number of branch edge executions. After a seed is executed, we traverse the edges that have been explored, and when a branch edge is found, add one to the number of executions of the edge on the statistics table. Note that no matter how many times a branch is executed by the same seed, the number of executions of the branch edge in the statistic table is only increased by one.

According to the statistic table, we judge the state of each constraint and calculate its priority score. The priority score determines the priority of the concolic executor to solve the constraint. The correspondence between the state of the constraint and the priority score is also listed in Table I. If neither edge of a constraint is explored (i.e., in the *unreached* state), the priority score is 0. As mentioned earlier, solving constraints in the *unreached* state does not help the fuzzer to discover new paths, so we consider that the fuzzer does not get stuck on unreached constraints and does not need help from the concolic executor. We use a higher priority queue to store *unsolved* constraints and a lower priority queue to store *solved* constraints. This means that we prioritize solving constraints that are in the *unsolved* state.

For a constraint in the *unsolved* state, we calculate the priority score using the number of executions of the explored edge. Specifically, the number of executions for the explored edge of an unsolved constraint is equal to the number of times the constraint has been explored by certain seeds. If a constraint is explored multiple times, but not solved, it means that the constraint can be complex for fuzzing. Besides, if a constraint is explored more frequently, it means that the constraint is in a critical position and many paths need to pass through it. Therefore, the more times the explored edge of an unsolved constraint is executed, the higher the priority of solving the constraint is. Priority score is calculated by

$$score = \sum_{i=1}^n T(seed_i) + \sum_{i=1}^n F(seed_i), \quad (1)$$

where n represents the number of seeds that have been executed. $T(seed_i)$ determines whether $seed_i$ explores the True branch of the constraint. If it does, $T(seed_i)$ is 1. Otherwise, $T(seed_i)$ is 0. $F(seed_i)$ determines the False branch. If explored, $F(seed_i)$ is 1, if not, $F(seed_i)$ is 0. Specifically,

if a constraint only has its True branch explored, then the priority of the constraint is the cumulative execution count of the True branch. Similarly, the priority of the constraint that only the False branch is explored is the cumulative execution count of the False branch. All constraints in the *unsolved* state are sorted in a queue (denoted as Q_{high}) according to their priority scores. The concolic executor selects the constraint that has highest priority score from Q_{high} . In order to prevent starvation, that is, the concolic executor always selects the same constraint to solve, Q_{high} also considers the number of times the constraints are selected when sorting constraints.

For a constraint in the *solved* state, we compute the priority score based on the rarity of the paths reachable by the constraint. Specifically, the difference in the number of times two branches of the same constraint are explored reflects the rarity of the paths. Suppose there is a constraint whose True branch has been explored for only one time by all seeds while its False branch has been explored for 100 times. It is clear that the True branch of this constraint is a rare path. Generating a seed that can explore the True branch of the constraint can increase the probability of triggering bugs of the target program [15]. Therefore, the priority score of the constraint in the *solved* state is computed by

$$score = \frac{|\sum_{i=1}^n T(seed_i) - \sum_{i=1}^n F(seed_i)|}{\sum_{i=1}^n T(seed_i) + \sum_{i=1}^n F(seed_i)}, \quad (2)$$

where $\sum_{i=1}^n T(seed_i)$ and $\sum_{i=1}^n F(seed_i)$ refer to the numbers of True and False branch executions, respectively. The absolute value of the difference between the number of executions of the two branches is the numerator, and the sum of the number of executions of the two branches is the denominator. The ratio of the two values is defined as the priority score. All constraints in the *solved* state are sorted in a lower priority queue (denoted as Q_{low}) according to priority score. Because solving constraints in the *unsolved* state can more directly improve the probability of finding vulnerabilities, we pick constraints from Q_{low} when there is no constraint in Q_{high} .

C. System Design

Figure 3 illustrates the system design of SILK including four components (i.e., static analyzer, fuzzer, coordinator and concolic executor), and the overall workflow of constraint-guided hybrid fuzzing that mainly contains the following four steps:

- 1) Before starting fuzzing, the static analyzer analyzes the information of the constraints in the target program. The analysis results contain two parts: one is the branch edge score, which represents the difficulty of exploring the paths reachable by each branch edge of the constraint. The other one is the constraint distribution information, which represents the code locations of these constraints. The branch edge scores are stored in the fuzzer to calculate the seed scores in the power scheduling algorithm, while the constraint distribution information is stored in the coordinator to construct the statistic table of constraint execution states.

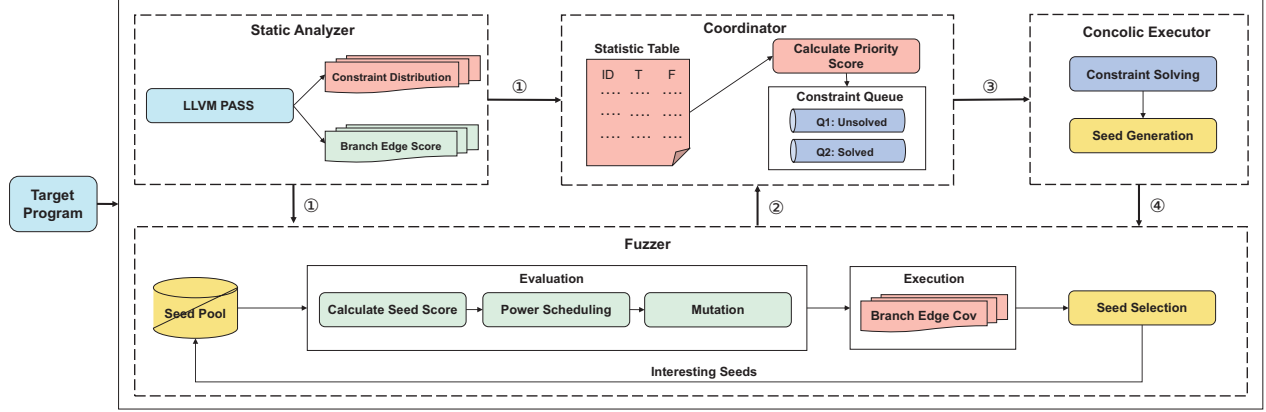


Fig. 3. System overview of SILK. Inputs are the target program and initial seeds (in the seed pool). SILK contains four components. (1)The Static analyzer conducts program analysis. (2)The fuzzer tests the target program and obtains the execution information of seeds. (3)The coordinator launches the concolic executor and uses the dynamic and static information of the constraints to select the seeds that are transmitted to the concolic executor. (4)The concolic executor solves the constraints and generates new seeds for fuzzing.

- 2) In the fuzzing phase, the fuzzer selects initial seeds from the seed pool and executes them to obtain execution information. Then, according to the execution information and the branch edge scores obtained in the static analysis phase, we use the proposed evolutionary algorithm to calculate seed scores and allocate power. Thus, new seeds are generated and those benefiting to discover new paths or new constraints will be appended in the seed pool. Meanwhile, the code coverage of all newly generated test cases is given to the coordinator for building the statistic table.
- 3) The coordinator uses the code coverage information of the test cases to count the number of executions of each branch of the constraints. The statistics are used to identify the execution state of the constraints and to calculate the constraint scores. Constraints are sorted into queues based on their state, and each queue sorts the constraints by priority score. When the concolic executor is idle, the coordinator picks the constraint with the highest priority from the constraint queue and transfers the seeds that can reach the constraint to the concolic executor.
- 4) The concolic executor performs constraint solving on the incoming seeds and generates seeds that can discover new paths. The new seeds will be saved to the seed pool in the fuzzer and the process of (2)(3)(4) will be repeated.

V. EVALUATION

SILK uses the Whole-Program-LLVM (WLLVM) [16] tool to compile the target program on which static analyzer is to be applied to a single LLVM bitcode module for analysis [17]. Then we wrote an LLVM pass to analyze the bitcode file to obtain the branch edge score and constraint distribution information. The fuzzing component of SILK is based on AFL. On top of AFL, we not only implement the path exploration

difficulty-based evolutionary algorithm but also monitor the execution of branch edges and collect data in a shared memory. The coordinator of SILK is implemented in Python, and its main functions include constructing a statistics table, reading the execution information of branch edges from the shared memory, calculating priority scores for constraints, constructing seed queues, and passing seeds that require constraint solving to the concolic executor. For lightweight and efficiency considerations, SILK uses a fast concolic execution engine, called QSYM [5], to support hybrid fuzzing.

In this section, we evaluate both the efficiency of SILK and the effectiveness of the proposed techniques. We attempt to answer the following questions:

- **RQ1:** Whether the hybrid fuzzing is more efficient with the help of two newly proposed techniques?
- **RQ2:** Whether the proposed evolutionary algorithm improves the efficiency of fuzzing to explore constraints?
- **RQ3:** Whether the proposed interaction strategy make concolic execution more effective in assisting fuzzing?

A. Evaluation Setup

We run our experiments on a server with eight Intel Core T7700 processors and 16 GB memory running 64-bit Ubuntu 16.04 LTS. Considering the diversity and complexity, we have carefully selected the programs shown in Table II, which covers five functionality types including: image, audio, text, binary and network packet processing software [18] as real-world benchmark programs. When selecting comparison objects we need to ensure that hybrid fuzzers have the same concolic executor. The mainstream concolic executors are Angr [19], KLEE [20], QSYM [5], and Intrigue [7], and they differ significantly in the effectiveness of constraint solving. If a hybrid fuzzer with a different concolic executor is chosen for comparison, such as Savior [21], Meuzz [22], the difference in performance between concolic executors can lead to unfair evaluation. In addition to this, our proposed technique can

be superimposed on other work, such as Pangolin [23], so that hybrid fuzzers that are compatible with our techniques need not be used as a comparison object. We compared SILK with the following fuzzers: AFL [24], QSYM [5], Driller [1], QuickFuzz [2]. AFL is employed as the baseline of coverage-based fuzzing while QSYM is considered as the baseline of concolic execution. Driller is the state-of-the-art hybrid fuzzer which adopts the “demand launch” strategy. QuickFuzz is the state-of-the-art hybrid fuzzer which adopts the “path priority” strategy. Besides, to answer RQ2, we took AFL with the addition of our proposed evolutionary algorithm (denoted as AFL-SILK) as another competitor. Since the open-source version of Driller uses Angr as the concolic executor and QuickFuzz is not open source, we implemented these two hybrid fuzzers ourselves based on QSYM.

TABLE II
REAL-WORLD BENCHMARK PROGRAMS AND EVALUATION ARGUMENTS.
@@ REPRESENTS THE INPUT FILE.

Type	Program	Version	Arguments
Image	tiff2ps	libtiff 4.3.0	@ @
	tiff2pdf	libtiff 4.3.0	@ @
	djpeg	libjpeg 9c	@ @
Audio	mp3gain	1.6.2	@ @
Text	sqlite3	3.38.1	(stdin)
Binary	objdump	binutils 2.38	-D @ @
	readelf	binutils 2.38	-a @ @
Network	tcpdump	5.0.0-PRE-GIT	-e -vv -nr @ @

This evaluation is divided into three parts. First, we use path coverage to represent the overall effect of fuzzers. Because if a fuzzer can discover more paths, it has a higher probability of triggering abnormal states of the target program. Then, we compare the branch coverage of different fuzzers. Branch coverage is a metric which represents the ability of fuzzers to discover and pass constraints that can answer RQ2. Finally, we analyze the quality of new seeds generated by the concolic executor under different interaction strategies to answer RQ3. To be fair, we assign three CPU cores to each fuzzer, run each experiment for 12 hours and repeat 5 times.

B. Path Coverage

We use the path coverage to represent the overall efficiency of fuzzers. In each round of experiments, we give each fuzzer exactly the same resources, including initial seeds, number of CPU cores and time. Following the suggestion of George et al [25], we use Mann-Whitney U-test [26] to measure the importance of our improvements. Figure 4 shows the number of paths triggered by different fuzzers on eight real-world programs over 12 hours. Overall, SILK finds more paths on most of the programs and has a clear lead in the first four hours. As an example, when testing *tiff2ps*, SILK finds 25% and 21% more paths in the first four hours than AFL and QSYM, respectively, and boosts the effect to 41% and 30% in the final results. The experimental results fully demonstrate that we have achieved the objective of improving

the path coverage at the early stage of testing and increasing the effectiveness of hybrid fuzzing. In addition, the low p-values (< 0.05) of Mann-Whitney U-test well support that our improvements are statistically significant. However, our work still has some limitations. From Figure 4, we can see that SILK has no advantage when testing *map3gain* and *sqlite3*. *Map3gain* is a very small program roughly one-tenth the size of *tiff2ps*, meaning that the additional overhead of analyzing the constraints will outweigh the benefits when testing programs with few constraints. The slow execution of seeds by *sqlite3* causes the coordinator to not have enough samples when counting the execution state of constraints, which causes poor overall results. Addressing these two limitations is our future work.

C. Branch Coverage

Our proposed techniques focus on constraints, so branch coverage is one of the most concerning metrics. We use a tool called AFL-COV [27] to count the branch coverage of each fuzzer, and the results are shown in Table III. AFL finds about 4% more constraints on average than the original with the help of our proposed evolutionary algorithm. SILK achieves the highest branch coverage on all programs except *map3gain* and improves it by about 4% over QSYM. The results show that under the same constraint solving ability, our proposed evolutionary algorithm can explore more constraints, in other words, can make full use of the constraint solving ability. Due to the small code size of *map3gain*, all fuzzers have approximately the same constraint coverage.

TABLE III
COMPARISON OF BRANCH COVERAGE

Fuzzer	AFL	AFL-SILK	QSYM	Driller	QuickFuzz	SILK
tiff2ps	3238	3387	3380	3288	3404	3530
tiff2pdf	4395	4379	4336	4399	4354	4518
djpeg	1735	1756	2014	1919	2021	2065
mp3gain	488	485	494	490	485	487
sqlite3	14901	15071	15670	14276	15720	15858
objdump	3978	4504	4809	4335	4901	4999
readelf	6192	6285	6007	6213	6364	6859
tcpdump	10627	11451	11601	10501	11598	11908
Average	5694	5915	6039	5677	6105	6278

D. Seed Generation Efficiency

In fuzzing, seeds that can discover new paths are called interesting seeds. The more interesting seeds that concolic execution can generate per unit time means that concolic execution is more helpful for fuzzing. Driller starts the concolic executor only when fuzzing gets stuck, causing the concolic executor to be idle for a long time. To ensure the fairness of the evaluation, we replace Driller with the native QSYM, thus ensuring that the execution time of concolic executors is the same. Note that Driller and native QSYM are identical except for the execution time of concolic executors. The number of interesting seeds generated by the concolic executor in

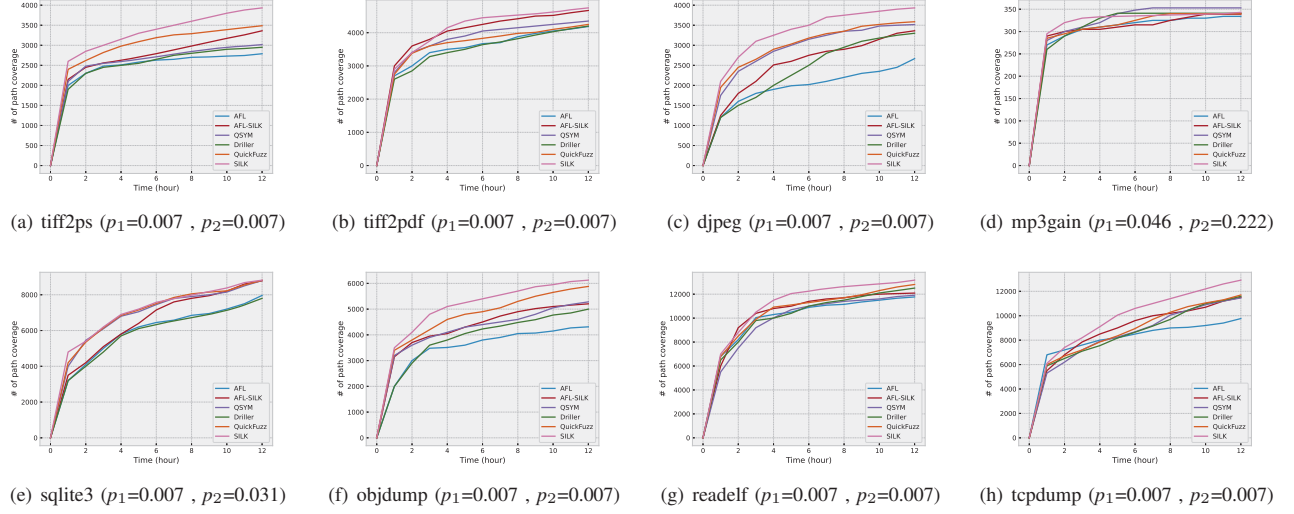


Fig. 4. Comparison of path coverage. p_1 and p_2 are the p-values for the Mann-Whitney U-test of SILK compared with AFL and QSYM respectively.

QSYM, QuickFuzz and SILK are shown in Figure 5. SILK can generate 1.5 to 3 times more interesting seeds than QSYM and more than 1 times more interesting seeds than QuickFuzz in 12 hours, fully illustrating the effectiveness of our proposed interaction strategy. On *sqlite3*, SILK does not work well for the same reason mentioned in Section V-B. We will optimize it in our future work.

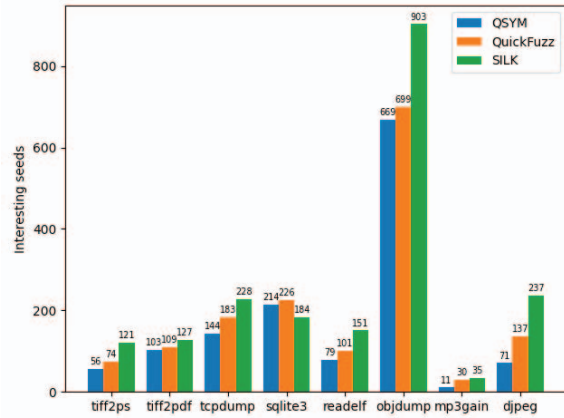


Fig. 5. Number of interesting seeds generated by concolic executors.

VI. RELATED WORK

A. Hybrid Fuzzing

Majundar et al. [28] introduce the idea of hybrid concolic testing which counteracts the shortcomings of random testing and concolic execution. Driller [1] combines fuzzing and concolic execution with “demand launch” strategy. In order to improve the efficiency of the interaction between fuzzing and concolic execution, DigFuzz [4] employs the Monte Carlo

based probabilistic path prioritization model and QuickFuzz [2] designs a model which is based on path lengths and sample hits. Both of these researches use the newly proposed models to sort the paths passed to the concolic execution. QSYM [5] develops a fast concolic execution engine which integrates the symbolic emulation with the native execution using dynamic binary translation. In addition, there are some studies that introduce new techniques to improve hybrid fuzzing, such as sanitizer [21], dynamic taint analysis [7], [29], machine learning [22]. Berry [30] proposes a sequence-directed hybrid fuzzing aiming to improve the ability to reproduce vulnerabilities.

B. Directed Greybox Fuzzing

AFLGo [13] and SemFuzz [31] are the first proposed directed greybox fuzzing systems that guide the seeds to the target locations by calculating distances. Hawkeye [32] modifies the distance definition which can reflect the call trace. ParmeSan [33] reduces time to target locations by introducing dynamic control flow graphs. CAFL [34] considers the ordered target sites and the data conditions which aim to satisfy a sequence of constraints. The purpose of our work is different from that of the existing directed greybox fuzzing. AFLGo and related work typically target vulnerable functions with the aim of reproducing vulnerabilities. We target unsolved, simple constraints to take advantage of the high throughput of fuzzing. To the best of our knowledge, we are the first to improve the adaptation of fuzzing to the characteristics of hybrid fuzzing.

VII. CONCLUSION

In this paper, we presented a constraint-guided hybrid fuzzing approach and implemented a prototype called SILK. We proposed in SILK two novel techniques, named *path exploration difficulty-based evolutionary algorithm* and *constraints execution state-guided interaction strategy*. The evolutionary

algorithm guides fuzzing to prioritize exploration of simple paths and the interaction strategy guides the concolic executor to first solve the constraints most likely to get stuck. Experimental results demonstrate the state of the art of our proposed techniques. SILK improved path coverage by 10%-45% and branch coverage by 5%-10% on real-world programs compared with other fuzzers. In future work, more precise information of constraints can be obtained to improve both the fuzzer and the coordinator, while the concolic executor can also be allocated to solve more concrete constraints.

VIII. ACKNOWLEDGMENT

We thank the anonymous reviewers for their helpful feedback. This project was supported by Chinese National Key Laboratory of Science and Technology on Information System Security (Grant No. 6142111210203), and the Fundamental Research Funds for the Central Universities (Grant No. 2242023K30034).

REFERENCES

- [1] N. Stephens, J. Grosen, C. Salls, A. Dutcher, R. Wang, J. Corbetta, Y. Shoshitaishvili, C. Kruegel, and G. Vigna, "Driller: Augmenting fuzzing through selective symbolic execution," in *Proceedings of the 2016 Network and Distributed System Security Symposium (NDSS)*, (San Diego, California, USA), pp. 1–16, Internet Society, 2016.
- [2] P. Lin, Z. Hong, Y. Li, and L. Wu, "A priority based path searching method for improving hybrid fuzzing," *Computers & Security*, vol. 105, p. 102242, 2021.
- [3] X. Wang, J. Sun, Z. Chen, P. Zhang, J. Wang, and Y. Lin, "Towards optimal concolic testing," in *Proceedings of the 40th International Conference on Software Engineering (ICSE)*, (New York, NY, USA), p. 291–302, ACM, 2018.
- [4] L. Zhao, Y. Duan, H. Yin, and J. Xuan, "Send hardest problems my way: Probabilistic path prioritization for hybrid fuzzing," in *Proceedings of the 2019 Network and Distributed System Security Symposium (NDSS)*, (San Diego, California, USA), Internet Society, 2019.
- [5] I. Yun, S. Lee, M. Xu, Y. Jang, and T. Kim, "QSYM : A practical concolic execution engine tailored for hybrid fuzzing," in *Proceedings of the 27th USENIX Security Symposium (USENIX Security)*, (Baltimore, MD), pp. 745–761, USENIX Association, Aug. 2018.
- [6] J. Li, B. Zhao, and C. Zhang, "Fuzzing: a survey," *Cybersecurity*, vol. 1, no. 1, pp. 1–13, 2018.
- [7] M. Cho, S. Kim, and T. Kwon, "Intriguer: Field-level constraint solving for hybrid fuzzing," in *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, (New York, NY, USA), p. 515–530, ACM, 2019.
- [8] Y. Li, S. Ji, C. Lyu, Y. Chen, J. Chen, Q. Gu, C. Wu, and R. Beyah, "V-fuzz: Vulnerability prediction-assisted evolutionary fuzzing for binary programs," *IEEE Transactions on Cybernetics*, vol. 52, no. 5, pp. 3745–3756, 2022.
- [9] DARPA, "Cyber grand challenge challenge repository," 2017. <http://www.lungetech.com/cgc-corpus/>.
- [10] P. Chen and H. Chen, "Angora: Efficient fuzzing by principled search," in *Proceedings of the 2018 IEEE Symposium on Security and Privacy (SP)*, pp. 711–725, 2018.
- [11] X. Mi, B. Wang, Y. Tang, P. Wang, and B. Yu, "Shfuzz: Selective hybrid fuzzing with branch scheduling based on binary instrumentation," *Applied Sciences*, vol. 10, no. 16, 2020.
- [12] S. Gan, C. Zhang, P. Chen, B. Zhao, X. Qin, D. Wu, and Z. Chen, "GREYONE: Data flow sensitive fuzzing," in *Proceedings of the 29th USENIX Security Symposium (USENIX Security)*, pp. 2577–2594, USENIX Association, 2020.
- [13] M. Böhme, V.-T. Pham, M.-D. Nguyen, and A. Roychoudhury, "Directed greybox fuzzing," in *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, (New York, NY, USA), p. 2329–2344, ACM, 2017.
- [14] H. Peng, Y. Shoshitaishvili, and M. Payer, "T-fuzz: fuzzing by program transformation," in *Proceedings of the 2018 IEEE Symposium on Security and Privacy (SP)*, pp. 697–710, 2018.
- [15] S. Rawat, V. Jain, A. Kumar, L. Cojocar, C. Giuffrida, and H. Bos, "Vuzzer: Application-aware evolutionary fuzzing," in *Proceedings of the 2017 Network and Distributed System Security Symposium (NDSS)*, vol. 17, pp. 1–14, 2017.
- [16] Tristan Ravitch, "Whole-program-llvm," 2019. <https://github.com/travitch/whole-program-llvm>.
- [17] Z. Wang, B. Liblit, and T. Reps, "Tofu: Target-oriented fuzzer," 2020.
- [18] Y. Li, S. Ji, Y. Chen, S. Liang, W.-H. Lee, Y. Chen, C. Lyu, C. Wu, R. Beyah, P. Cheng, K. Lu, and T. Wang, "UNIFUZZ: A holistic and pragmatic Metrics-Driven platform for evaluating fuzzers," in *Proceedings of the 30th USENIX Security Symposium (USENIX Security)*, pp. 2777–2794, USENIX Association, 2021.
- [19] Y. Shoshitaishvili, R. Wang, C. Hauser, C. Krügel, and G. Vigna, "Firmalace - automatic detection of authentication bypass vulnerabilities in binary firmware," in *Network and Distributed System Security Symposium*, 2015.
- [20] C. Cadar, D. Dunbar, and D. Engler, "Klee: Unassisted and automatic generation of high-coverage tests for complex systems programs," in *Proceedings of the 8th USENIX Conference on Operating Systems Design and Implementation (OSDI)*, p. 209–224, USENIX Association, 2008.
- [21] Y. Chen, P. Li, J. Xu, S. Guo, R. Zhou, Y. Zhang, T. Wei, and L. Lu, "Savior: Towards bug-driven hybrid testing," in *Proceedings of the 2020 IEEE Symposium on Security and Privacy (SP)*, pp. 1580–1596, 2020.
- [22] Y. Chen, M. Ahmadi, R. M. Farkhani, B. Wang, and L. Lu, "MEUZZ: Smart seed scheduling for hybrid fuzzing," in *Proceedings of the 23rd International Symposium on Research in Attacks, Intrusions and Defenses (RAID)*, (San Sebastian), pp. 77–92, USENIX Association, 2020.
- [23] H. Huang, P. Yao, R. Wu, Q. Shi, and C. Zhang, "Pangolin: Incremental hybrid fuzzing with polyhedral path abstraction," in *2020 IEEE Symposium on Security and Privacy (SP)*, pp. 1613–1627, 2020.
- [24] Michał Zalewski, "american fuzzy lop," 2014. <https://lcamtuf.coredump.cx/afl/>.
- [25] G. Klees, A. Ruef, B. Cooper, S. Wei, and M. Hicks, "Evaluating fuzz testing," in *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, pp. 2123–2138, 2018.
- [26] P. E. McKnight and J. Najab, "Mann-whitney u test," *The Corsini encyclopedia of psychology*, pp. 1–1, 2010.
- [27] "Afl fuzzing code coverage," 2017. <https://github.com/mrash/afl-cov>.
- [28] R. Majumdar and K. Sen, "Hybrid concolic testing," in *Proceedings of the 29th International Conference on Software Engineering (ICSE)*, pp. 416–426, 2007.
- [29] T. Wang, T. Wei, G. Gu, and W. Zou, "Taintscope: A checksum-aware directed fuzzing tool for automatic software vulnerability detection," in *Proceedings of the 2010 IEEE Symposium on Security and Privacy (SP)*, pp. 497–512, 2010.
- [30] H. Liang, L. Jiang, L. Ai, and J. Wei, "Sequence directed hybrid fuzzing," in *Proceedings of the 2020 IEEE 27th International Conference on Software Analysis, Evolution and Reengineering (SANER)*, pp. 127–137, 2020.
- [31] W. You, P. Zong, K. Chen, X. Wang, X. Liao, P. Bian, and B. Liang, "Semfuzz: Semantics-based automatic generation of proof-of-concept exploits," in *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, (New York, NY, USA), p. 2139–2154, Association for Computing Machinery, 2017.
- [32] H. Chen, Y. Xue, Y. Li, B. Chen, X. Xie, X. Wu, and Y. Liu, "Hawkeye: Towards a desired directed grey-box fuzzer," in *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, (New York, NY, USA), p. 2095–2108, Association for Computing Machinery, 2018.
- [33] S. Österlund, K. Razavi, H. Bos, and C. Giuffrida, "ParmeSan: Sanitizer-guided greybox fuzzing," in *Proceedings of the 29th USENIX Security Symposium (USENIX Security)*, pp. 2289–2306, USENIX Association, 2020.
- [34] G. Lee, W. Shim, and B. Lee, "Constraint-guided directed greybox fuzzing," in *Proceedings of the 30th USENIX Security Symposium (USENIX Security)*, pp. 3559–3576, USENIX Association, 2021.